Тема 3. Взаимодействие «родственных» процессов и нитей

.

3.1. Теоретическое введение

Функционирование любой ЭВМ заключается в выполнении строго определённой последовательности действий, называемой **программой** или **задачей**. «*Запустить программу на выполнение*» означает загрузить саму программу (написанную на понятном для ЭВМ языке) и необходимые для неё данные в оперативную память и настроить процессор (значения его внутренних регистров) так, чтобы он стал выполнять инструкции, начиная с соответствующей ячейки оперативной памяти.

В мультизадачных системах, к которым относится и ОС Linux, в оперативную память может быть загружено сразу несколько программ. При этом, чередуя выполнение на одном процессоре инструкций то одной программы, то другой, ЭВМ создаёт «иллюзию» их одновременного выполнения. Правила, по которым чередуются инструкции разных программ, определяются операционной системой (подсистема планирования процессов или нитей). При этом программы ничего не знают о существовании друг друга[1].

В многопроцессорных (многоядерных) системах каждый процесс может выполняться на собственном процессоре (ядре). При этом каждый процессор, в свою очередь, может использоваться для выполнения нескольких (псевдо)параллельных процессов.

Для того чтобы операционная система могла определить, инструкцию какой программы следует выполнить следующей, должны выполняться два условия: во-первых, после выполнения очередной инструкции одной из программ процессор должен переключиться на операционную систему[2], и, во-вторых, операционная система должна обладать информацией обо всех «выполняющихся» программах (место в оперативной памяти, где располагаются инструкции программы; номер последней выполненной инструкции; значения регистров процессора после выполнения последней инструкции и т.п.). Информация обо всех выполняющихся программах хранится в оперативной памяти вместе с операционной системой.

Совокупность, состоящая из программы, описывающей её информации и дополнительных служебных данных, называется **процессом**. Информация о программе хранится в операционной системе в виде специальной структуры, называемой **дескриптором процесса**. Дескриптор процесса и дополнительная информация о программе вместе называются **контекстом процесса**.

---

[1] Следует помнить, что защита программ реализуется как на аппаратурном уровне (защищённый режим процессора), так и на уровне операционной системы.

[2] Напомним, что для ЭВМ операционная система является обычной программой, также загруженной в оперативную память.

Идея многопроцессных (многопрограммных) операционных систем заключается в том, что на одной ЭВМ (псевдо)одновременно исполняется несколько программ (процессов)[3]. При этом считается, что процессы: 1) независимы и 2) ничего не знают о существовании друг друга[4]. Каждому процессу выделяется свой набор ресурсов, которыми он монопольно владеет, и никто другой (кроме операционной системы) не имеет прямого доступа к ним.

Каждый процесс, в свою очередь, может выполнять несколько нитей (потоков инструкций). При этом все нити одного процесса также выполняются независимо друг от друга и обладают некоторым набором «собственных» ресурсов, но без труда могут получить доступ к общей памяти процесса.

Очевидно, что нити изначально предназначены для взаимодействия друг с другом. А могут ли возникнуть ситуации, когда одному процессу необходимо «сообщить» другому (или нескольким другим) процессу(-ам) какую-либо информацию? Ответ очевиден – да. С такими взаимодействиями мы уже сталкивались, когда изучали конвейерный способ запуска программ. Кроме этого, процессу, например, может потребоваться:

- сообщить о своей готовности обработать данные;
- «поделиться» с другим процессом объемом выполняемой работы или взять часть работы другого процесса на себя;
- обеспечить строгую последовательность работы нескольких процессов (например, чтобы начисление процентов на счете в банке производилось только после его пополнения);
- и т.п.

К средствам межпроцессного взаимодействия (англ. Interprocess Communications, IPC), реализованных в операционной системе Linux, относятся:

- ожидание завершения родственного процесса;
- сигналы;
- каналы (именованные и неименованные);
- очереди сообщений;
- семафоры;
- разделяемая память;
- сокеты.

Последний способ IPC позволяет реализовать взаимодействие между процессами, выполняющимися на разных ЭВМ, соединённых каналами передачи данных.

Все средства IPC условно можно разделить по типу процессов, участвующих во взаимодействии:

---

[3] Очевидно, что в случае многопроцессорной (многоядерной) ЭВМ процессы на самом деле выполняются параллельно.

[4] Вспомним, что каждый процесс «знает» лишь номер своего родительского процесса (поле PPID дескриптора).

– «любой процесс». Ресурсы IPC доступные всем процессам (согласно правам доступа). Такие ресурсы создаются и управляются самими процессами. К этому типу IPC относятся: именованные каналы, очереди сообщений, семафоры, разделяемая память;
– «родственные процессы». Такие ресурсы создаются и управляются операционной системой. К этому типу ресурсов относятся: сигналы, неименованные каналы, сокеты.

3.2. Порождение «дочернего» процесса

Новый «родственный» процесс порождается с помощью системного вызова **fork**:

```
#include <sys/types.h>
#include <unistd.h>

pid_t fork(void);
```

Порождённый или *дочерний процесс* является точной копией родительского процесса за исключением следующих атрибутов:

- идентификатор процесса (PID);
- идентификатор родительского процесса (PPID). У дочернего процесса он приравнивается к PID родительского процесса (того, кто выполнил вызов *fork*);
- дочерний процесс не имеет очереди сигналов, ожидающих обработки.

После завершения вызова *fork*, оба процесса начинают выполнять одну и ту же инструкцию – следующую за вызовом *fork*.

Чтобы процесс мог определить, кем он является (дочерним или родительским), используется возвращаемое значение вызова *fork*. В первом случае, когда процесс является дочерним, *fork* вернёт 0, во втором – PID вновь созданного процесса. Если *fork* возвращает –1, то это свидетельствует об ошибке (естественно, в этом случае возврат происходит только в процесс, выполнивший системный вызов, и новый процесс не создаётся).

Пример программы, порождающей новый процесс, приведён в листинге 6.

Листинг 6. Порождение нового процесса

```
1. #include <stdio.h>
2. #include <unistd.h>
3.
4. int main(void)
5. {
6.   int pid;
7.   pid = fork();
8.   if (pid == -1)
```

Листинг 6. Порождение нового процесса

```
9.   {
10.    perror(”fork”); exit(1);
11.  }
12.  if (pid == 0)
13.  {
14.    /*Эта часть кода выполняется дочерним процессом*/
15.    printf(”Потомок\n”);
16.    sleep(1);
17.  }
18.  else
19.  {
20.    /*Эта часть кода выполняется родительским
21.      процессом*/
22.    printf(”Родитель. Создан потомок %u\n”, pid);
23.    sleep(1);
24.  }
25.  return (0);
26.}
```

В строках 1-2 подключаются необходимые заголовочные файлы. В данном случае нам требуются функции ввода-вывода (*stdio.h*) и функции по работе с процессами (*unistd.h*). Порождение процесса происходит в строке 7, после чего проверяется возвращаемое значение функции *fork*. Если новый процесс создан, то условие в строке 8 не выполнится, и каждый процесс перейдёт на строку 12. Далее, проверив значение переменной *pid*, дочерний процесс будет выполнять строки 13-17 (так как в его случае *pid* = 0), а родительский – строки 19-24. Затем оба процесса завершатся с кодом 0 (строка 25).

Особое внимание следует обратить на строки 16 и 22, в которых родительский и дочерний процессы добровольно переходят в «спящее» состояние на 1 секунду. Для чего это сделано? Ответ прост – для того чтобы оба процесса смогли вывести информацию в поток вывода, и она была отображена на экране терминала. Если этого не сделать, то может возникнуть ситуация, когда один из процессов (в силу алгоритмов планирования) завершит свою работу быстрее, чем другой начнёт выводить информацию в поток вывода. В этом случае завершившийся процесс автоматически закроет все открытые файлы, которыми в нашем случае являются потоки ввода-вывода. В силу того, что файловые дескрипторы у родственных процессов одинаковые, ещё не завершившийся процесс начнёт выдавать информацию в поток, у которого соответствующий файл уже закрыт. И на экран никакой информации выдано не будет.

3.3. Неименованные (программные) каналы для взаимодействия «родственных» процессов

Вспомните синтаксис организации программных каналов при работе в командной строке *shell*:

```
$cat myfile | sort
```

При этом (стандартный) вывод программы *cat*, которая выводит содержимое файла *myfile*, передаётся на (стандартный) ввод программы *sort*, которая, в свою очередь, отсортирует предложенный материал.

Таким образом, два процесса обменялись данными. При этом использовался программный канал, обеспечивающий однонаправленную передачу данных между двумя задачами.

Для создания канала в программе используется системный вызов **pipe**.

```
#include <unistd.h>

int pipe (int *filedes);
```

Этот вызов возвращает два файловых дескриптора: *filedes*[1] для записи в канал и *filedes*[0] для чтения из канала. Теперь, если один процесс записывает данные в *filedes*[1], то другой сможет получить эти данные из *filedes*[0]. Вопрос только в том, как другой процесс сможет получить сам файловый дескриптор *filedes*[1]? Вспомним наследуемые атрибуты при создании процесса. Дочерний процесс наследует и разделяет все назначенные файловые дескрипторы родительского. То есть доступ к дескрипторам *filedes* канала может получить сам процесс, вызвавший *pipe*, и его дочерние процессы. В этом заключается серьёзный недостаток каналов, поскольку они могут быть использованы для передачи данных только между родственными процессами. Каналы не могут использоваться в качестве средства межпроцессного взаимодействия между независимыми процессами.

Для работы с каналом `pipe` используются системные вызовы `write` и `read` соответственно для записи и чтения данных в канал и из канала. Прототипы этих функций имеют следующий вид:

```
ssize_t write( int fd,const void *buf, size_t count);

ssize_t read( int fd, const void *buf,  size_t count);
```

где fd – файловый дескриптор канала, *buf – указатель на буферную переменную, из которой или в которую будет производиться запись или чтение данных в канал или из канала, count- количество байт записываемых

или считываемых из канала. (для определения количества информации в байтах можно воспользоваться функцией `sizeof(имяпеременной)`.

Lab3.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <string.h>

void process(char *s) {
Обработка содержимого файла
}


int main() {
int fd[2];
pipe(fd);
pid_t pid = fork();
if (pid > 0) {
// родительский процесс
close(fd[0]); // закрываем чтение
                FILE *f = fopen("input.txt", "r");
                    if (!f) { perror("input"); exit(1); }
                  char c;
                   while ((c = fgetc(f)) != EOF) // чтение символа из файла
write(fd[1], &c, 1);                //запись в канал
c = 26;
write(fd[1], &c, 1);
close(fd[1]);
fclose(f);
}
else {
// дочерний процесс
close(fd[1]); // закрываем запись в канал
char buf[5000];
int len = 0;
3
char c;
while (read(fd[0], &c, 1) == 1) { // читаем из канала
if ((unsigned char)c == 26) break;
buf[len++] = c;
}
buf[len] = '\0';
```

```
process(buf);  // обработка прочитанного из канала текста
FILE *out = fopen("output.txt", "w"); // открытие нового файла для записи информации
fputs(buf, out); // запись в файл обработанной информации
fclose(out); // закрытие нового файла
close(fd[0]); // закрытие канала
}
return 0;
}.
```

Практическая работа №3.

Взаимодействие «родственных» процессов.

Задание на практическую работу

Написать программу на языке Си,

выполняющую следующие действия:

- создание программного канала и порождение дочернего процесса;
- **основной процесс считывает содержимое файла , *(создайте файл с помощью текстового редактора)* , и** передаёт его в канал; концом передачи данных в канал сделать код 26.
- дочерний процесс получает информацию через канал и при получении информации дочерний процесс производит работу согласно заданию по варианту из таблицы 3.1
- выводит эту информацию в файл с новым именем
- работа дочернего процесса завершается передачей имени нового файла родителю.

Таблица 3.1

Вариант	Условие задачи
1	Из заданного тексте, начиная с первой встретившейся буквы ‘а’, переписать в новый массив все символы до первой встретившейся буквы ‘ к’.
2	В исходном тексте удалить все символы пробела, создав из оставшихся символов новый массив. Вывести преобразованный текст и количество удаленных пробелов.
3	Определить, сколько раз в заданном тексте встречается сочетание двух первых символов.

4	В качестве исходного предложения задать арифметическое выражение, записанное на языке С. Проверить, соблюдается ли баланс открывающихся и закрывающихся скобок. Если равенство соблюдается, то вывести соответствующее сообщение, в противном случае вывести количество открывающихся и закрывающихся скобок.
5	Создать новый текст из гласных букв исходного.
6	В заданном тексте найти слово, в котором максимальное количество буквы ‘а’. Вывести сколько раз буква ‘а’ встречается в этом слове.
7	Сформировать числовой массив N, элементы которого указывают длину каждого слова в исходном тексте.
8	Сформировав новый массив В из исходного текста, поставив перед каждым словом знак ‘?’,
9	Составить новый текст, содержащий первые буквы каждого слова исходного.
10	В каждом слове заданного текста поменять местами первую букву и последнюю.
11	Указать, сколько раз первый символ первого слова встречается в заданном тексте.
12	В заданном тексте заменить все запятые на «!», не создавая нового массива. Вывести преобразованный текст, а также количество удаленных запятых.
13	В заданном тексте заменить все строчные латинские буквы прописными.
14	Определить, есть ли в заданном тексте цифры, посчитать сколько их.
15	Создать новый текст из исходного, поставив в новом в конце каждого слова точку и каждое слово записать с большой буквы.
16	Собрать новый текст из последних символов слов исходного текста.
17	В заданном тексте поменять местами последнюю букву предыдущего слова с первой буквой следующего слова.
18	Сформировать числовой массив N, элементы которого указывают количество гласных букв в каждом слове исходного предложения.
19	Сформировать числовой массив N, элементы которого

	указывают количество букв ‘о’ в каждом слове исходного предложения.